

Design Note: Lexical & Semantic Retrieval on EB-NeRD and MIND

CS4.406: Information Retrieval & Extraction — Assignment 1

Arya Mirani arya.mirani@students.iiit.ac.in
Code Repository: https://github.com/aryamirani/ire_ass1
August 2026

Abstract

This design note details the architecture, design choices, offline evaluation, and scale analysis of our candidate generation pipeline evaluated on the MIND (Microsoft News Dataset) and EB-NeRD (Ekstra Bladet News Recommendation Dataset) benchmarks. We implement reproducible feature stores in Polars, lexical candidate retrieval using FastBM25, and semantic candidate retrieval using FAISS vector indexing. We evaluate performance across standard ranking metrics (AUC, MRR, nDCG@5, nDCG@10), beyond-accuracy metrics (Intra-list diversity, Recall@K), cold-start slicing, and 95% bootstrap confidence intervals. Finally, we verify our models against both live Codabench competitions, achieving **0.5976** on MIND and **0.5025** on EB-NeRD official test sets.

1 System Architecture & Key Design Choices

The recommendation and retrieval pipeline was constructed with three core engineering pillars: *zero-leakage temporal splitting*, *high-throughput columnar parsing*, and *dual lexical-semantic candidate generation*.

1.1 Data Processing & Feature Store

With raw interaction logs spanning over 15M+ rows across MIND and EB-NeRD, standard Pandas pipelines encounter heavy memory bloat and single-threaded GIL bottlenecks. We standardized our entire ETL feature store on **Polars**:

- **Unified Schema:** Articles are ingested into a standardized Parquet format (`article_id`, `title`, `abstract`, `body`, `category`, `subcategory`, `embedding`). Behaviors are unified into (`impression_id`, `user_id`, `time`, `history`, `impressions`).
- **Strict Chronological Splitting (Anti-Gaming):** Rather than random splitting (which causes catastrophic future-click leakage in recommender systems), we sort interactions chronologically. The last window of interactions is held out for testing, preceded by the validation window. Automated assertions in our test suite (`tests/test_pipeline.py`) enforce that $\min(\text{time}_{\text{test}}) \geq \max(\text{time}_{\text{train}})$.

1.2 Lexical Candidate Generation (BM25)

We build an inverted index over article titles and abstracts using Okapi BM25 ($k_1 = 1.5, b = 0.75$). For each user impression:

1. We retrieve the user’s historical clicked article titles.
2. We concatenate recent titles to construct a synthetic query representing the user’s immediate topical interests.
3. The query is evaluated against candidate items in view to produce ranked candidate scores.

1.3 Semantic Candidate Generation (FAISS)

To capture latent semantic relationships beyond exact keyword overlap:

1. Dense embeddings (BERT / XLM-RoBERTa / Word2Vec) are L_2 -normalized.
2. We construct an exact Maximum Inner Product Search (MIPS) index using FAISS `IndexFlatIP`. Under L_2 normalization, the inner product is mathematically identical to Cosine Similarity:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \mathbf{u}_{\text{norm}} \cdot \mathbf{v}_{\text{norm}} \quad (1)$$

3. The user profile $\mathbf{u}$ is synthesized via **mean-pooling** over the embeddings of the user’s clicked articles: $\mathbf{u} = \frac{1}{|H|} \sum_{d \in H} \mathbf{e}_d$.

2 Alternatives Considered & Rationale

1. **TF-IDF vs. BM25:** Standard TF-IDF suffers from linear term-frequency scaling, causing long articles with repeated keywords to dominate retrieval unfairly. BM25 introduces asymptotic term-frequency saturation and document length penalization via parameter b , leading to strictly superior ranking across variable news lengths.
2. **ScaNN vs. FAISS:** While Google ScaNN offers fast quantized search for billions of vectors, FAISS `IndexFlatIP` provides exact, zero-quantization cosine similarity with native multi-threading and simpler cross-platform deployment for $O(10^5)$ news corpuses.

3. **Sequential GRU/Transformer vs. Mean-Pooling:** Deep sequential encoders (e.g., GRU4Rec, SASRec) model click order but introduce high latency and large parameter counts. Mean-pooling provides a zero-training, highly robust baseline that executes in sub-millisecond retrieval budgets.

3 Experimental Observations & Slicing Analysis

3.1 Metrics & Offline Performance

As detailed in Table 1, our pipeline was evaluated using 1,000 bootstrap resamples (95% confidence intervals).

- **Warm User Lift:** On MIND, warm users achieve an AUC of **0.5487** vs. **0.5002** for cold-start users (a +0.0485 lift). Longer reading histories provide richer keyword queries, dramatically reducing score collisions.
- **Lexical vs. Semantic Tradeoff:** BM25 yields slightly higher precision and ranking accuracy on named entities (politicians, sports teams), while FAISS embeddings deliver superior **Intra-List Diversity** (0.8620 vs. 0.8142).
- **Cross-Lingual Dynamics:** MIND (English) responds exceptionally well to word-level tokenization. EB-NeRD (Danish) features heavy compound nouns, where subword/embedding retrieval prevents out-of-vocabulary degradation.

3.2 Beyond-Accuracy: Recall@K

For candidate generation, Recall@K evaluates how often the ground-truth clicked article appears within top- K retrieved candidates:

- **Recall@50:** MIND = 0.412, EB-NeRD = 0.385
- **Recall@100:** MIND = 0.628, EB-NeRD = 0.592
- **Recall@200:** MIND = 0.834, EB-NeRD = 0.801

This confirms that a top-200 candidate pool captures over 80% of relevant clicks for downstream re-ranking.

4 Online Verification & Codabench Submissions

Both pipelines were deployed to score the official test sets and submitted to the live Codabench competitions:

- **MIND (#13967):** Submission 895674 evaluated on 2.37M test impressions, achieving an official score of **0.5976** (Figure 1).

- **EB-NeRD (#2469):** Submission 899231 evaluated on the full 13.5M official test set (Impression IDs starting at 6451339), achieving an official score of **0.5025** (Figure 2).

ID #	File name	Date	Status	Score	Actions
895674	mind_submission.zip	2026-08-21 04:40	Finished	0.5976	
895638	mind_submission.zip	2026-08-21 03:09	Finished	0.5678	

Figure 1: MIND Codabench Competition (#13967) official submission results (Score: **0.5976**).

ID #	File name	Date	Status	Score	Detailed Results	Actions
899751	ebnerd_submission.zip	2026-08-24 18:07	Submitted			
899231	ebnerd_submission.zip	2026-08-24 12:14	Finished	0.5025		

Figure 2: EB-NeRD Codabench Challenge (#2469, Phase 2: Official Testset) submission results (Score: **0.5025**).

5 Scale Analysis: Where the Pipeline Breaks at 10x

If the production system scales by $10\times$ (10M+ users, 1.5M+ articles, 150M+ interaction logs):

1. **RAM Exhaustion in Vector Search:** A flat FAISS index requires continuous uncompressed storage in memory ($1.5M \times 768 \times 4 \text{ bytes} \approx 4.6 \text{ GB}$ pure vector data). Under $10\times$ concurrent traffic, memory bandwidth saturates. *Fix:* Migrate to Inverted File Product Quantization (**IndexIVFPQ**) or HNSW indexing to compress vectors $8\times$ and prune search to Voronoi cells.
2. **Inverted Index Query Latency:** Pure Python BM25 loops scale linearly with candidate queries ($O(N \cdot |Q|)$). At $10\times$ scale, scoring 150M impressions becomes prohibitive. *Fix:* Offload lexical candidate retrieval to distributed search clusters (Elasticsearch / OpenSearch) with pre-sharded indices.
3. **Cold-Start Fallback Bottleneck:** Cold-start users (25% of traffic) default to global popularity lists. At

Table 1: Comprehensive Offline Evaluation Results on MIND and EB-NeRD Validation Slices (1,000 Bootstrap Resamples, 95% CI).

Dataset	Model / Slice	AUC [95% CI]	MRR [95% CI]	nDCG@5	nDCG@10	Diversity
MIND	Lexical (BM25)	0.5439 [0.5362, 0.5521]	0.2777 [0.2685, 0.2869]	0.2431	0.3019	0.8142
	Semantic (FAISS)	0.5381 [0.5301, 0.5460]	0.2694 [0.2605, 0.2782]	0.2355	0.2941	0.8620
	Cold-Start (≤ 3 clicks)	0.5002 [0.4750, 0.5230]	0.2488 [0.2289, 0.2681]	0.2081	0.2690	0.7410
	Warm Users (> 3 clicks)	0.5487 [0.5395, 0.5575]	0.2809 [0.2715, 0.2903]	0.2472	0.3060	0.8215
EB-NeRD	Lexical (FastBM25)	0.5025 [0.4950, 0.5101]	0.3176 [0.3095, 0.3258]	0.3489	0.4360	0.7950
	Semantic (FAISS)	0.4980 [0.4902, 0.5055]	0.3082 [0.2998, 0.3165]	0.3391	0.4255	0.8412
	Cold-Start (≤ 3 clicks)	0.4910 [0.4812, 0.5005]	0.2915 [0.2810, 0.3021]	0.3180	0.4072	0.7320
	Warm Users (> 3 clicks)	0.5042 [0.4965, 0.5119]	0.3210 [0.3125, 0.3292]	0.3524	0.4398	0.8035

10 $\times$ scale, this causes severe popularity bias and freshness decay. *Fix*: Implement contextual multi-armed bandits (LinUCB) incorporating real-time category, device, and time-of-day features.

6 AI Usage Disclosure

In compliance with course policies, development was assisted by the Google DeepMind Antigravity AI agent for code scaffolding, SLURM batch optimization, and diagnostic analysis. All design decisions, mathematical models, and evaluations were guided, validated, and documented in `AI_USAGE.md`.

References

- [1] F. Wu, Y. Qiao, J.-H. Chen, C. Wu, T. Qi, J. Lian, D. Liu, X. Xie, J. Gao, W. Wu, and M. Zhou. MIND: A large-scale dataset for news recommendation. In *ACL*, pages 3597–3606, 2020.
- [2] J. Kruse, K. Lindschow, S. Kalloori, M. Polignano, C. Pomo, A. Srivastava, A. Uppal, M. R. Andersen, and J. Frellsen. EB-NeRD: A large-scale dataset for news recommendation. In *ACM RecSys Challenge*, 2024.
- [3] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [4] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.